

Laboratorio Nro. 3

Conversor Analógico Digital

Comisión Tauro - Aguirre

Actividad 1: Sensor digital de luminosidad

Determinar la resistencia del LDR a partir del valor digital del ADC:

El valor digital entregado por el ADC se convierte a LUX en dos pasos. Primero, se calcula el voltaje medido por el ADC usando la relación:

$$V_{adc} = \frac{ADCvalue}{1023} * V_{ref}$$

Luego, considerando que el LDR está en un divisor de tensión con una resistencia fija se calcula la resistencia del LDR:

$$R_{ldr} = R * (\frac{V_{ref}}{V_{adc}} - 1), \text{ con } R = 1k \text{ ohms.}$$

Finalmente, para convertir R_{ldr} a LUX se realiza una interpolación lineal entre los valores de la tabla de calibración, la cual relaciona resistencia del LDR con luminosidad. Este método permite obtener una estimación precisa de la luminosidad para cualquier valor de resistencia dentro del rango medido, asegurando un registro continuo de los valores de luz, incluyendo el promedio, mínimo, máximo y porcentaje dentro del rango dinámico.

Resistencia R	LUX
92000	0.5
41000	1
24000	3
16000	6
10000	10
7000	15
5000	35
1000	80
500	100

Estimar la frecuencia de muestreo necesaria para obtener los valores utilizados en el cálculo de la luminosidad promedio.

El tiempo entre muestras lo calculamos teniendo en cuenta que vamos a tener 200 muestras en un lapso de tiempo de 15 segundos, entonces:

$$T_{muestreo} = \frac{15s}{200} = 0.075 s = 75 ms$$

Esto significa que se debe tomar una muestra cada 0.075s

Luego debemos convertirla a frecuencia de muestreo:

$$Fs = \frac{1}{T_{muestreo}} = \frac{1}{0.075} = 13.33Hz$$

Para que el sistema pueda calcular correctamente la luminosidad promedio usando 200 muestras en 15 segundos, la frecuencia de muestreo del ADC debe ser de aproximadamente 13 Hz.

Identificar las tareas del sistema e indicar para cada una los dispositivos asociados (display, pulsadores, ADC, etc).

Tarea del sistema	Dispositivo(s) asociado(s)	Descripción
Lectura de luminosidad actual	ADC, LDR	El ADC convierte el voltaje entregado por el LDR en un valor digital. Se utiliza un callback para procesar la conversión y calcular la resistencia y LUX.
Almacenamiento de valores en buffer circular	Memoria interna (RAM)	Guarda las últimas 200 muestras de LUX para poder calcular promedio, mínimo, máximo y porcentaje del rango dinámico.
Cálculo de luminosidad promedio, mínimo y máximo	Microcontrolador (CPU)	Procesa los datos del buffer circular para actualizar los valores de LUX promedio, mínimo y máximo.
Cálculo de porcentaje de luminosidad en el rango min-max	Microcontrolador (CPU)	Calcula el porcentaje relativo del valor actual de LUX dentro del rango dinámico registrado.

Selección de qué valor mostrar (actual, mínimo, máximo, promedio)	Pulsadores (teclado del laboratorio 2)	Permite al usuario seleccionar qué valor de LUX visualizar en la pantalla.
Visualización de luminosidad	Display / Serial	Muestra los valores de LUX seleccionados (actual, promedio, mínimo o máximo) en un display o por Serial hacia una app de monitoreo.
Monitoreo y comunicación con la app	UART / Serial	Envía las tramas con los valores de LUX para ser visualizados en Processing o cualquier otra interfaz externa.

Determinar los eventos que iniciarán la ejecución de cada una de las tareas identificadas.

Tarea del sistema	Evento que la inicia
Lectura de luminosidad actual	Disparo realizado por el timer cada 75 ms
Almacenamiento de valores en buffer circular	Lectura de un nuevo valor de LUX desde el ADC
Cálculo de luminosidad promedio, mínimo y máximo	Inserción de un nuevo valor en el buffer (cada vez que se agrega una muestra)
Cálculo de porcentaje de luminosidad en el rango min-max	Actualización de LUX actual (después del cálculo de promedio/min/max)
Selección de qué valor mostrar (actual, mínimo, máximo, promedio)	Pulsación de un botón por el usuario en el teclado
Visualización de luminosidad	Cambio en el valor seleccionado por el usuario o actualización de LUX (trigger interno)
Monitoreo y comunicación con la app	Cada vez que se obtiene un nuevo valor de LUX (ADC callback)

Consideraciones de implementación de los drivers:

adc_driver.cpp

El ADC se utiliza para digitalizar señales analógicas provenientes de dos fuentes distintas:

- **Canal 0:** teclado analógico (mediciones cada 10 ms).
- **Canal 1:** sensor LDR (mediciones cada 75 ms).

El ADC está configurado en single mode y no en free running porque necesitamos controlar cuándo y qué canal se mide: el Timer1 dispara una conversión en el canal 1 (LDR) cada 75 ms, mientras que el Timer2 hace lo mismo en el canal 0 (teclado) cada 10 ms. Así, cada fuente tiene su propio ritmo de muestreo y no se superponen conversiones; al terminar cada una, la ISR del ADC guarda el valor y ejecuta el callback correspondiente.

Decidimos implementar el ADC con dos timers independientes ya que solo necesitamos muestrear dos dispositivos, el teclado y el LDR, con frecuencias distintas. Esta solución es simple y directa, aunque también resulta en un excesivo uso de recursos de hardware. Entendemos que si el proyecto se ampliara a más canales, lo adecuado sería tener un único timer que gestione los cambios de canal y las conversiones en round robin, obteniendo así un sistema que constantemente está procesando lo que lee el ADC y en donde se delega la responsabilidad a los drivers de almacenar la información en los periodos de tiempo precisados.

ldr_driver.cpp

La implementación del driver para el sensor LDR está basado en la medición del voltaje intermedio en un divisor resistivo, convertido a valores de LUX mediante una tabla de interpolación. Mediante la función `resistance_to_lux()` se interpola linealmente entre los valores de la tabla, mostrada más arriba en este informe.

Respecto a la tabla, cabe destacar que se utiliza la dada en las consignas de este Laboratorio, pero con ciertas adaptaciones mínimas para que los valores en casos de luz ambiente sean intermedios, en un rango entre 0 y 100 LUX. Los valores de resistencia de la tabla original fueron interpretados en k ohms.

Cada lectura se toma cada 75 ms, luego es procesada, ingresada en un buffer circular de 200 espacios y evaluada para recalcular valores mínimos, máximos y promedio en un intervalo de aproximadamente 15 segundos.

El driver también incluye la emisión de datos por Serial en formato de trama (LUX actual, máximo, mínimo, promedio), permitiendo la integración con una aplicación de visualización personalizada en Processing.

teclado_driver.cpp

El driver del teclado está implementado sobre un único canal del ADC, aprovechando un divisor resistivo que genera distintos valores de tensión según la tecla presionada.

El software compara la lectura digitalizada con una tabla de umbrales para identificar cada tecla. Se incluye un mecanismo de antirrebote por software contador `db_counter`, para filtrar lecturas falsas y solo registrar cambios estables de estado. El driver soporta detección de eventos de key down y key up mediante callbacks registrados por el usuario, lo que permite desacoplar la lógica de hardware de la aplicación principal.

main.cpp

En el `setup()` se inicializan todos los periféricos y drivers: la comunicación serie, el LCD, el LDR con su callback de actualización y el teclado, además de la cola de funciones. También se registran los handlers para cada tecla, de modo que al presionar una se cambie el modo de visualización LUX actual, promedio, máximo o mínimo.

El `loop()` principal queda reducido a ejecutar `fnqueue_run()`, que procesa las funciones encoladas por las interrupciones ADC.

Las funciones auxiliares cumplen roles específicos:

- **on_ldr_update()** recibe el valor de LUX actualizado, refresca la variable global y dispara la actualización del display cada cierto intervalo.
- **update_lcd_display()** se encarga de mostrar en el LCD el valor según el modo actual normal, promedio, máximo o mínimo.
- **on_keyboard_down_event()** interpreta qué tecla se presionó y cambia el modo de visualización del LCD en consecuencia.
- **process_serial_commands()** para permitir que el sistema reciba comandos desde el host a través del puerto serie. Esta función lee del buffer de Serial si hay algo disponible para leer y lo interpreta para cambiar el modo de visualización del LCD:
 - **1** → medición de LUX actual
 - **2** → máximo valor medido
 - **3** → mínimo valor medido
 - **4** → promedio

Código processing:

Imports y declaración de variables:

El sketch importa la librería *processing.serial.** para permitir la comunicación con Arduino vía Serial. Se declaran variables para graficar funciones que representan el LUX actual, promedio, máximo y mínimo. También se definen botones para cambiar el modo de visualización, labels para mostrar los valores y el modo activo, y variables de control como *modo_grafico*, *en_modos_individual* y *buttonPressed* para gestionar qué gráfico mostrar y la interacción del usuario. Finalmente, se definen variables de tamaño de ventana y viewports para organizar la interfaz, y *myPort* para la conexión con Arduino.

setup():

Esta función se ejecuta una sola vez al iniciar el sketch. Configura la ventana de Processing con *size(1000, 400)*, establece la fuente y prepara los gráficos *ScrollingFcnPlot*. Crea los botones rectangulares y las etiquetas de texto para mostrar valores de LUX y el modo activo. Inicializa el puerto Serial para recibir datos, usando un buffer hasta el carácter de nueva línea.

Botones:

Todos los botones funcionan de la misma forma: si se presionan una vez, muestran únicamente el gráfico correspondiente al botón elegido; si el mismo botón se vuelve a presionar, el sistema vuelve a mostrar todos los gráficos simultáneamente.

- **Botón 1 – “MEDICION” / LUX Actual**

Este botón controla la visualización del LUX actual. Al presionarlo se muestra solo el gráfico de la medición actual y el sistema cambia *modo_grafico* a 1. Si se vuelve a presionar, retorna a *modo_grafico* = 0 y se muestran todos los gráficos. Se envía el comando **1** al Arduino para reflejar el cambio y mostrar la medición actual en el LCD.

- **Botón 2 – “MAX” / LUX Máximo**

Este botón activa la visualización del LUX máximo registrado. Al presionarlo se muestra solo ese gráfico, cambiando *modo_grafico* a 2. Si se presiona nuevamente, el sistema retorna a *modo_grafico* = 0 mostrando todos los gráficos. El comando **2** es enviado al Arduino y se muestra el valor máximo medido en los últimos 15 segundos en el LCD.

- **Botón 3 – “MIN” / LUX Mínimo**

Este botón controla la visualización del LUX mínimo registrado. Al presionarlo, el sistema muestra únicamente ese gráfico, cambiando *modo_grafico* = 3. Al presionarlo de nuevo, vuelve a *modo_grafico* = 0 y se muestran todos los gráficos. Se envía el comando **3** al Arduino, lo que muestra en el LCD el valor mínimo dentro de los últimos 15 segundos.

- **Botón 4 – “PROM” / LUX Promedio**

Este botón alterna la visualización del LUX promedio. Al presionarlo una vez se muestra solo el gráfico de promedio, poniendo `modo_grafico = 4`. Si se vuelve a presionar, retorna a `modo_grafico = 0` mostrando todos los gráficos simultáneamente. Se envía el comando **4** al Arduino para que se muestre el valor promedio calculado, tanto en valores de LUX como de porcentaje.

Archivos del repositorio:

Directorio *deploy*

Directorio para archivos de aplicaciones. Incluye el datasheet del sensor LDR y el subdirectorio de la aplicación Processing (SE_Lab_HostTempApp). Dentro de este se encuentra el archivo principal de Processing para la aplicación de visualización, el archivo de código para botones, el código que define la clase de las etiquetas y el que define los gráficos.

Directorio *doc*

Se incluyen el informe del Laboratorio y el documento con las consignas del Laboratorio 3.

Directorio *include*

Se incluyen los archivos header para cada cpp correspondiente:

- **adc_driver.h**: header para el driver del ADC
- **critical.h**: header para secciones críticas
- **fnqueue.h**: header para la cola de funciones
- **ldr_driver.h**: header para el driver del LDR
- **teclado_driver.h**: header para el driver del teclado

Directorio *src*

Contiene los archivos cpp con todo el código para el correcto funcionamiento del programa:

- **adc_driver.cpp**: implementación del driver del ADC
- **critical.cpp**: implementación de secciones críticas
- **fnqueue.cpp**: implementación de la cola de funciones
- **ldr_driver.cpp**: implementación del driver del LDR
- **main.cpp**: archivo principal del programa Arduino
- **teclado_driver.cpp**: implementación del driver del teclado